ATIVIDADE AVALIATIVA

Banco de Dados - Comandos SQL Oracle

Fintech SignalInvest

Integrantes do grupo
José Eduardo Silva de Azevedo - RM 99642
Marcello Loureiro Costa - RM 570621
Mario Henrique Silverio de Souza - RM 573501
Eduardo Vieira Da Silva - RM 571602
Efraim Martins Buriti - RM 562586

Oracle Database | Modelagem relacional evoluída | DML e consultas

1. Objetivo e aderência aos requisitos

Este documento apresenta a solução da atividade avaliativa de Banco de Dados do projeto fintech SignalInvest. A entrega foi estruturada para Oracle Database e utiliza como base a modelagem relacional da Fase 3, evoluída apenas onde necessário para representar conta bancária, receitas, despesas e investimentos diretamente vinculados ao usuário.

Requisito	Como foi atendido
Banco Oracle	Tipos VARCHAR2, NUMBER, DATE/TIMESTAMP e funções TO_DATE/SYSTIMESTAMP.
Modelo relacional	Inclusão do modelo físico da Fase 3 e do recorte relacional evoluído usado nos comandos.
Cadastro	5 comandos INSERT: usuário, conta bancária, receita, despesa e investimento.
Alteração	4 comandos UPDATE usando os IDs exigidos para restringir o registro correto.
Consultas simples	Usuário, despesa e investimento por seus códigos.
Consultas ordenadas	Despesas e investimentos do usuário, do mais recente ao mais antigo.
Dashboard	Consulta única que retorna uma linha com o usuário, sua última despesa e seu último investimento.
Máscaras	Valores dinâmicos expressos com [CAMPO] e textos entre aspas simples, conforme o enunciado.

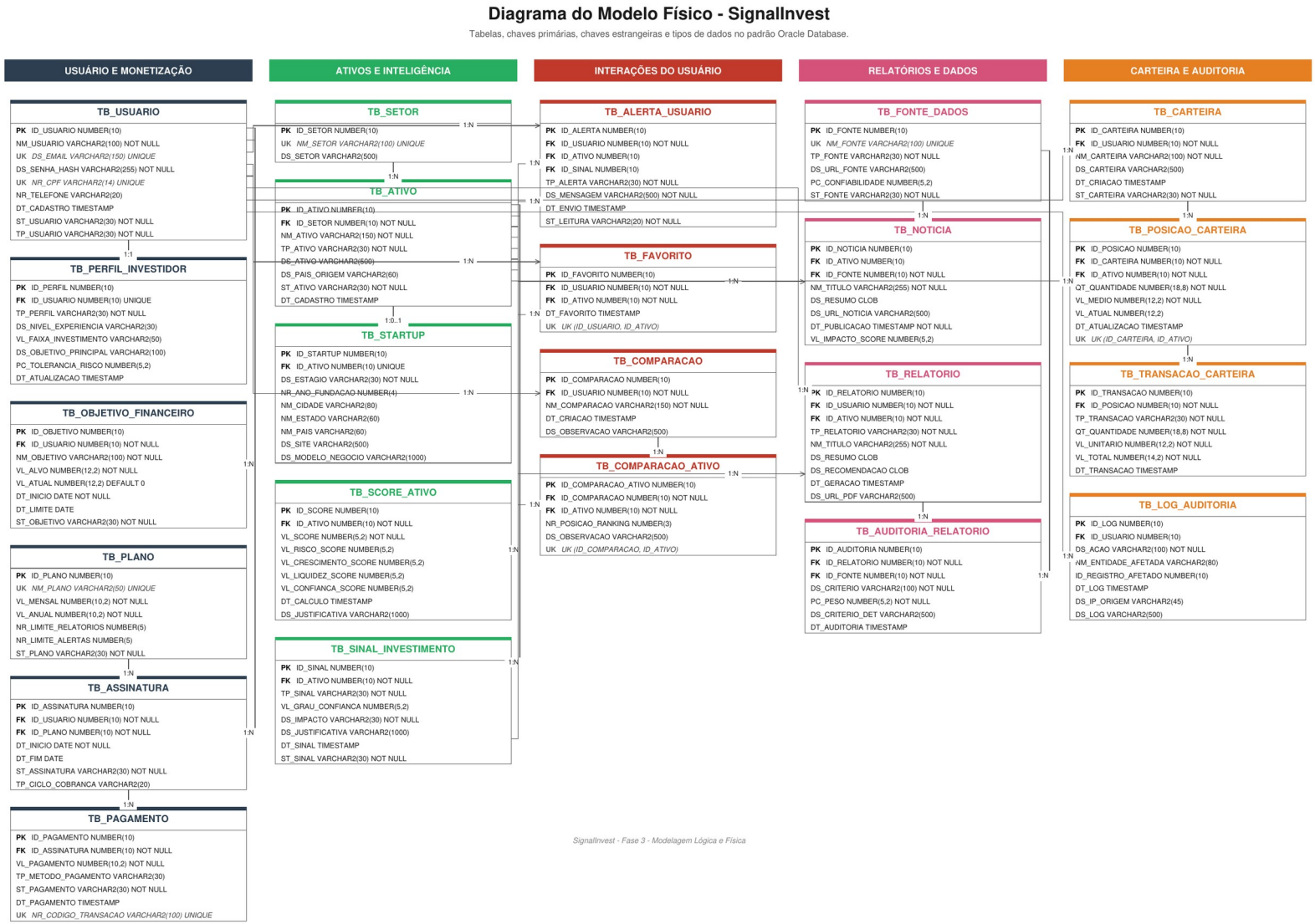
2. Evolução da modelagem para esta atividade

A modelagem original da SignalInvest já contém TB_USUARIO, TB_ATIVO, carteira, posições e transações de carteira. Para atender de forma direta e inequívoca às operações solicitadas no enunciado, o modelo foi evoluído com quatro entidades financeiras operacionais: TB_CONTA_BANCARIA, TB_RECEITA, TB_DESPESA e TB_INVESTIMENTO. Essa evolução mantém o tema fintech e preserva as entidades centrais da Fase 3.

A tabela TB_INVESTIMENTO liga o usuário a um ativo da plataforma e registra o aporte de forma diretamente consultável pelo par ID_USUARIO + ID_INVESTIMENTO, exatamente como solicitado na atividade. A carteira da modelagem original continua representando a visão consolidada de posições do usuário.

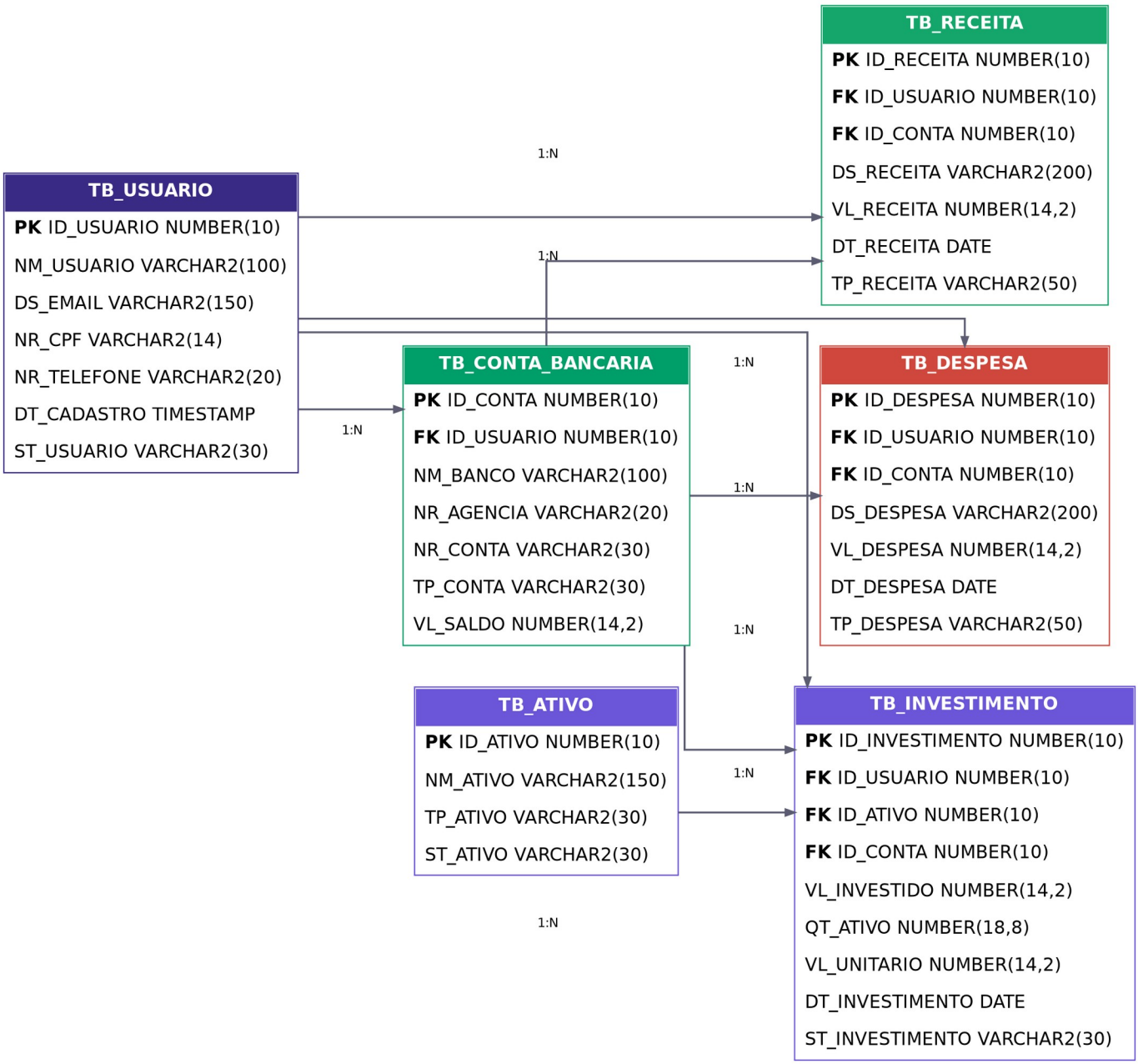
Modelo relacional físico - Fase 3 (base do projeto)

Diagrama original fornecido pelo grupo, com tabelas, PKs, FKs e tipos de dados Oracle.



Modelo relacional evoluído - recorte utilizado nesta atividade

As novas entidades financeiras se relacionam com TB_USUARIO; TB_INVESTIMENTO também referencia TB_ATIVO. **O modelo evoluído também aplica restrições de integridade para valores positivos e consistência entre usuário e conta bancária.**



3. Estrutura das tabelas adicionadas

Tabela	Campos principais	Finalidade
TB_CONTA_BANCARIA	ID_CONTA (PK), ID_USUARIO (FK), NM_BANCO, NR_AGENCIA, NR_CONTA, TP_CONTA, VL_SALDO	Conta bancária vinculada ao usuário.
TB_RECEITA	ID_RECEITA (PK), ID_USUARIO (FK), ID_CONTA (FK), DS_RECEITA, VL_RECEITA, DT_RECEITA, TP_RECEITA	Entradas de dinheiro.
TB_DESPESA	ID_DESPESA (PK), ID_USUARIO (FK), ID_CONTA (FK), DS_DESPESA, VL_DESPESA, DT_DESPESA, TP_DESPESA	Gastos do usuário.
TB_INVESTIMENTO	ID_INVESTIMENTO (PK), ID_USUARIO (FK), ID_ATIVO (FK), ID_CONTA (FK), VL_INVESTIDO, QT_ATIVO, VL_UNITARIO, DT_INVESTIMENTO, ST_INVESTIMENTO	Aportes realizados em ativos da SignalInvest.

3.1 Integridade referencial, cardinalidades e restrições

O recorte evoluído mantém as chaves primárias e estrangeiras do modelo e explicita as cardinalidades utilizadas nesta atividade. As restrições abaixo reforçam a consistência dos dados diretamente no Oracle, além das validações realizadas pela futura aplicação Java/JDBC.

Relacionamento	Cardinalidade / regra
TB_USUARIO -> TB_CONTA_BANCARIA	1:N - um usuário pode possuir várias contas.
TB_USUARIO -> TB_RECEITA	1:N - um usuário pode registrar várias receitas.
TB_USUARIO -> TB_DESPESA	1:N - um usuário pode registrar várias despesas.
TB_USUARIO -> TB_INVESTIMENTO	1:N - um usuário pode realizar vários investimentos.
TB_CONTA_BANCARIA -> TB_RECEITA	1:N - a conta identifica a origem/destino financeiro da receita.
TB_CONTA_BANCARIA -> TB_DESPESA	1:N - a despesa fica vinculada à conta utilizada.

TB_CONTA_BANCARIA -> TB_INVESTIMENTO	1:N - o aporte registra a conta associada à operação.
TB_ATIVO -> TB_INVESTIMENTO	1:N - um ativo pode estar presente em vários investimentos.

Restrições adicionais do modelo evoluído: VL_RECEITA, VL_DESPESA, VL_INVESTIDO, QT_ATIVO e VL_UNITARIO devem ser maiores que zero quando aplicável. A combinação (ID_CONTA, ID_USUARIO) é única em TB_CONTA_BANCARIA e é reutilizada em chaves estrangeiras compostas, impedindo que uma receita, despesa ou investimento associe uma conta pertencente a outro usuário.

3.2 Scripts Oracle da evolução do modelo

Os comandos abaixo materializam no Oracle as quatro tabelas acrescentadas para esta etapa. As máscaras continuam reservadas aos comandos DML solicitados pelo enunciado; no DDL são definidos tipos, PKs, FKs, UNIQUE e CHECK constraints.

3.2.1 TB_CONTA_BANCARIA

```
CREATE TABLE TB_CONTA_BANCARIA (
    ID_CONTA       NUMBER(10)     NOT NULL,
    ID_USUARIO     NUMBER(10)     NOT NULL,
    NM_BANCO       VARCHAR2(100)  NOT NULL,
    NR_AGENCIA     VARCHAR2(20)   NOT NULL,
    NR_CONTA       VARCHAR2(30)   NOT NULL,
    TP_CONTA       VARCHAR2(30)   NOT NULL,
    VL_SALDO       NUMBER(14,2)   DEFAULT 0 NOT NULL,

    CONSTRAINT PK_TB_CONTA_BANCARIA
        PRIMARY KEY (ID_CONTA),
    CONSTRAINT FK_CONTA_USUARIO
        FOREIGN KEY (ID_USUARIO)
        REFERENCES TB_USUARIO (ID_USUARIO),
    CONSTRAINT UK_CONTA_USUARIO
        UNIQUE (ID_CONTA, ID_USUARIO)
);
```

3.2.2 TB_RECEITA

```
CREATE TABLE TB_RECEITA (
    ID_RECEITA     NUMBER(10)     NOT NULL,
    ID_USUARIO     NUMBER(10)     NOT NULL,
    ID_CONTA       NUMBER(10)     NOT NULL,
    DS_RECEITA     VARCHAR2(200)  NOT NULL,
    VL_RECEITA     NUMBER(14,2)   NOT NULL,
    DT_RECEITA     DATE           NOT NULL,
    TP_RECEITA     VARCHAR2(50)   NOT NULL,

    CONSTRAINT PK_TB_RECEITA PRIMARY KEY (ID_RECEITA),
    CONSTRAINT FK_RECEITA_USUARIO
        FOREIGN KEY (ID_USUARIO)
        REFERENCES TB_USUARIO (ID_USUARIO),
    CONSTRAINT FK_RECEITA_CONTA_USUARIO
        FOREIGN KEY (ID_CONTA, ID_USUARIO)
        REFERENCES TB_CONTA_BANCARIA (ID_CONTA, ID_USUARIO),
    CONSTRAINT CK_RECEITA_VALOR CHECK (VL_RECEITA > 0)
```

```
);
```

3.2.3 TB_DESPESA

```
CREATE TABLE TB_DESPESA (
    ID_DESPESA     NUMBER(10)     NOT NULL,
    ID_USUARIO     NUMBER(10)     NOT NULL,
    ID_CONTA       NUMBER(10)     NOT NULL,
    DS_DESPESA     VARCHAR2(200)  NOT NULL,
    VL_DESPESA     NUMBER(14,2)   NOT NULL,
    DT_DESPESA     DATE           NOT NULL,
    TP_DESPESA     VARCHAR2(50)   NOT NULL,

    CONSTRAINT PK_TB_DESPESA PRIMARY KEY (ID_DESPESA),
    CONSTRAINT FK_DESPESA_USUARIO
        FOREIGN KEY (ID_USUARIO)
        REFERENCES TB_USUARIO (ID_USUARIO),
    CONSTRAINT FK_DESPESA_CONTA_USUARIO
        FOREIGN KEY (ID_CONTA, ID_USUARIO)
        REFERENCES TB_CONTA_BANCARIA (ID_CONTA, ID_USUARIO),
    CONSTRAINT CK_DESPESA_VALOR CHECK (VL_DESPESA > 0)
);
```

3.2.4 TB_INVESTIMENTO

```
CREATE TABLE TB_INVESTIMENTO (
    ID_INVESTIMENTO  NUMBER(10)     NOT NULL,
    ID_USUARIO       NUMBER(10)     NOT NULL,
    ID_ATIVO         NUMBER(10)     NOT NULL,
    ID_CONTA         NUMBER(10)     NOT NULL,
    VL_INVESTIDO     NUMBER(14,2)   NOT NULL,
    QT_ATIVO         NUMBER(18,8)   NOT NULL,
    VL_UNITARIO      NUMBER(14,2)   NOT NULL,
    DT_INVESTIMENTO  DATE           NOT NULL,
    ST_INVESTIMENTO  VARCHAR2(30)   NOT NULL,

    CONSTRAINT PK_TB_INVESTIMENTO PRIMARY KEY (ID_INVESTIMENTO),
    CONSTRAINT FK_INVEST_USUARIO
        FOREIGN KEY (ID_USUARIO) REFERENCES TB_USUARIO (ID_USUARIO),
    CONSTRAINT FK_INVEST_ATIVO
        FOREIGN KEY (ID_ATIVO) REFERENCES TB_ATIVO (ID_ATIVO),
    CONSTRAINT FK_INVEST_CONTA_USUARIO
        FOREIGN KEY (ID_CONTA, ID_USUARIO)
        REFERENCES TB_CONTA_BANCARIA (ID_CONTA, ID_USUARIO),
    CONSTRAINT CK_INVEST_VALOR CHECK (VL_INVESTIDO > 0),
    CONSTRAINT CK_INVEST_QT CHECK (QT_ATIVO > 0),
    CONSTRAINT CK_INVEST_UNITARIO CHECK (VL_UNITARIO > 0)
);
```

3.3 Decisões de implementação

Identificadores (IDs). Nos INSERTs e demais comandos da atividade, os IDs continuam representados por máscaras como [CÓDIGO DO USUÁRIO], exatamente como solicitado no enunciado. Em uma implementação integrada com Java/JDBC, a geração desses identificadores pode ser automatizada no Oracle por IDENTITY ou SEQUENCE, sem alterar a lógica dos comandos apresentados.

Valor do investimento. TB_INVESTIMENTO mantém VL_INVESTIDO, QT_ATIVO e VL_UNITARIO. QT_ATIVO e VL_UNITARIO detalham a operação, enquanto VL_INVESTIDO registra o valor financeiro consolidado do aporte no momento da transação. Essa escolha preserva o valor efetivamente registrado e facilita consultas de dashboard e histórico.

Controle transacional. O COMMIT permanece fora dos comandos individuais porque, na integração com Java/JDBC, a aplicação deverá confirmar ou reverter a transação de forma atômica conforme o sucesso das operações.

4. Convenções utilizadas nos comandos SQL

- Os comandos seguem a nomenclatura física do projeto (prefixo TB_ e campos em maiúsculas).
- Valores de texto usam máscaras entre aspas simples, por exemplo '[NOME DO USUÁRIO]'. Valores numéricos usam máscaras sem aspas, por exemplo [CÓDIGO DO USUÁRIO].
- Datas recebidas como texto usam TO_DATE('[DATA]', 'DD/MM/YYYY'), garantindo conversão explícita no Oracle.
- O SYSTIMESTAMP é utilizado para a data de cadastro do usuário, pois o valor é gerado pelo próprio banco no momento da operação.
- Os comandos UPDATE sempre incluem o ID do registro e, quando exigido, também o ID_USUARIO, evitando alterações acidentais em outros registros.
- O COMMIT não foi incluído em cada comando porque, na integração com Java/JDBC, a transação deve ser confirmada ou revertida de forma controlada pela aplicação.

5. Cadastro

5.1 Cadastrar os dados de um novo usuário

Insere um novo usuário utilizando os campos definidos em TB_USUARIO.

```
INSERT INTO TB_USUARIO (
    ID_USUARIO, NM_USUARIO, DS_EMAIL, DS_SENHA_HASH,
    NR_CPF, NR_TELEFONE, DT_CADASTRO, ST_USUARIO, TP_USUARIO
) VALUES (
    [CÓDIGO DO USUÁRIO],
    '[NOME DO USUÁRIO]',
    '[E-MAIL DO USUÁRIO]',
    '[HASH DA SENHA]',
    '[CPF DO USUÁRIO]',
```

```
    '[TELEFONE DO USUÁRIO]',
    SYSTIMESTAMP,
    '[STATUS DO USUÁRIO]',
    '[TIPO DO USUÁRIO]'
);
```

5.2 Cadastrar os dados da conta bancária de um usuário

A conta fica vinculada ao usuário pela chave estrangeira ID_USUARIO.

```
INSERT INTO TB_CONTA_BANCARIA (
    ID_CONTA, ID_USUARIO, NM_BANCO, NR_AGENCIA,
    NR_CONTA, TP_CONTA, VL_SALDO
) VALUES (
    [CÓDIGO DA CONTA],
    [CÓDIGO DO USUÁRIO],
    '[NOME DO BANCO]',
    '[NÚMERO DA AGÊNCIA]',
    '[NÚMERO DA CONTA]',
    '[TIPO DA CONTA]',
    [VALOR DO SALDO]
);
```

5.3 Cadastrar os dados de uma nova receita

Registra uma entrada financeira e associa a receita ao usuário e à conta bancária.

```
INSERT INTO TB_RECEITA (
    ID_RECEITA, ID_USUARIO, ID_CONTA, DS_RECEITA,
    VL_RECEITA, DT_RECEITA, TP_RECEITA
) VALUES (
    [CÓDIGO DA RECEITA],
    [CÓDIGO DO USUÁRIO],
    [CÓDIGO DA CONTA],
    '[DESCRIÇÃO DA RECEITA]',
    [VALOR DA RECEITA],
    TO_DATE('[DATA DA RECEITA]', 'DD/MM/YYYY'),
    '[TIPO DA RECEITA]'
);
```

5.4 Cadastrar os dados de uma nova despesa

Registra um gasto do usuário, mantendo a origem da conta bancária.

```
INSERT INTO TB_DESPESA (
    ID_DESPESA, ID_USUARIO, ID_CONTA, DS_DESPESA,
    VL_DESPESA, DT_DESPESA, TP_DESPESA
) VALUES (
    [CÓDIGO DA DESPESA],
    [CÓDIGO DO USUÁRIO],
    [CÓDIGO DA CONTA],
    '[DESCRIÇÃO DA DESPESA]',
    [VALOR DA DESPESA],
    TO_DATE('[DATA DA DESPESA]', 'DD/MM/YYYY'),
    '[TIPO DA DESPESA]'
);
```

5.5 Cadastrar os dados de um novo investimento

O investimento relaciona o usuário, sua conta bancária e o ativo selecionado na SignalInvest.

```
INSERT INTO TB_INVESTIMENTO (
    ID_INVESTIMENTO, ID_USUARIO, ID_ATIVO, ID_CONTA,
    VL_INVESTIDO, QT_ATIVO, VL_UNITARIO,
    DT_INVESTIMENTO, ST_INVESTIMENTO
) VALUES (
    [CÓDIGO DO INVESTIMENTO],
    [CÓDIGO DO USUÁRIO],
    [CÓDIGO DO ATIVO],
    [CÓDIGO DA CONTA],
    [VALOR INVESTIDO],
    [QUANTIDADE DO ATIVO],
    [VALOR UNITÁRIO],
    TO_DATE('[DATA DO INVESTIMENTO]', 'DD/MM/YYYY'),
    '[STATUS DO INVESTIMENTO]'
);
```

6. Alteração

6.1 Alterar os dados de um usuário

O ID_USUARIO identifica unicamente o registro que deverá ser atualizado.

```
UPDATE TB_USUARIO
SET NM_USUARIO = '[NOVO NOME DO USUÁRIO]',
    DS_EMAIL = '[NOVO E-MAIL DO USUÁRIO]',
    NR_TELEFONE = '[NOVO TELEFONE DO USUÁRIO]',
    ST_USUARIO = '[NOVO STATUS DO USUÁRIO]',
```

```
    TP_USUARIO = '[NOVO TIPO DO USUÁRIO]'
WHERE ID_USUARIO = [CÓDIGO DO USUÁRIO];
```

6.2 Alterar uma receita já cadastrada

A condição utiliza simultaneamente o código do usuário e o código da receita.

```
UPDATE TB_RECEITA
SET ID_CONTA = [NOVO CÓDIGO DA CONTA],
    DS_RECEITA = '[NOVA DESCRIÇÃO DA RECEITA]',
    VL_RECEITA = [NOVO VALOR DA RECEITA],
    DT_RECEITA = TO_DATE('[NOVA DATA DA RECEITA]', 'DD/MM/YYYY'),
    TP_RECEITA = '[NOVO TIPO DA RECEITA]'
WHERE ID_USUARIO = [CÓDIGO DO USUÁRIO]
  AND ID_RECEITA = [CÓDIGO DA RECEITA];
```

6.3 Alterar uma despesa já cadastrada

O filtro composto protege contra atualização de uma despesa pertencente a outro usuário.

```
UPDATE TB_DESPESA
SET ID_CONTA = [NOVO CÓDIGO DA CONTA],
    DS_DESPESA = '[NOVA DESCRIÇÃO DA DESPESA]',
    VL_DESPESA = [NOVO VALOR DA DESPESA],
    DT_DESPESA = TO_DATE('[NOVA DATA DA DESPESA]', 'DD/MM/YYYY'),
    TP_DESPESA = '[NOVO TIPO DA DESPESA]'
WHERE ID_USUARIO = [CÓDIGO DO USUÁRIO]
  AND ID_DESPESA = [CÓDIGO DA DESPESA];
```

6.4 Alterar um investimento já cadastrado

O investimento é atualizado somente quando os dois identificadores correspondem.

```
UPDATE TB_INVESTIMENTO
SET ID_ATIVO = [NOVO CÓDIGO DO ATIVO],
    ID_CONTA = [NOVO CÓDIGO DA CONTA],
    VL_INVESTIDO = [NOVO VALOR INVESTIDO],
    QT_ATIVO = [NOVA QUANTIDADE DO ATIVO],
    VL_UNITARIO = [NOVO VALOR UNITÁRIO],
    DT_INVESTIMENTO = TO_DATE('[NOVA DATA DO INVESTIMENTO]', 'DD/MM/YYYY'),
    ST_INVESTIMENTO = '[NOVO STATUS DO INVESTIMENTO]'
WHERE ID_USUARIO = [CÓDIGO DO USUÁRIO]
  AND ID_INVESTIMENTO = [CÓDIGO DO INVESTIMENTO];
```

7. Consultas simples

7.1 Consultar os dados de um usuário específico

Consulta direta pela chave primária do usuário.

```
SELECT ID_USUARIO,
       NM_USUARIO,
       DS_EMAIL,
       NR_CPF,
       NR_TELEFONE,
       DT_CADASTRO,
       ST_USUARIO,
       TP_USUARIO
FROM TB_USUARIO
WHERE ID_USUARIO = [CÓDIGO DO USUÁRIO];
```

7.2 Consultar uma despesa específica

Utiliza o código do usuário e o código da despesa, conforme solicitado.

```
SELECT ID_DESPESA,
       ID_USUARIO,
       ID_CONTA,
       DS_DESPESA,
       VL_DESPESA,
       DT_DESPESA,
       TP_DESPESA
FROM TB_DESPESA
WHERE ID_USUARIO = [CÓDIGO DO USUÁRIO]
  AND ID_DESPESA = [CÓDIGO DA DESPESA];
```

7.3 Consultar um investimento específico

Consulta o aporte pelo usuário e pelo identificador do investimento.

```
SELECT I.ID_INVESTIMENTO,
       I.ID_USUARIO,
       I.ID_ATIVO,
       A.NM_ATIVO,
       I.ID_CONTA,
       I.VL_INVESTIDO,
```

```
       I.QT_ATIVO,
       I.VL_UNITARIO,
       I.DT_INVESTIMENTO,
       I.ST_INVESTIMENTO
FROM TB_INVESTIMENTO I
JOIN TB_ATIVO A
  ON A.ID_ATIVO = I.ID_ATIVO
WHERE I.ID_USUARIO = [CÓDIGO DO USUÁRIO]
  AND I.ID_INVESTIMENTO = [CÓDIGO DO INVESTIMENTO];
```

8. Consultas ordenadas

8.1 Consultar todas as despesas de um usuário, da mais recente à mais antiga

O desempate pelo ID_DESPESA deixa a ordenação determinística quando duas despesas possuem a mesma data.

```
SELECT ID_DESPESA,
       DS_DESPESA,
       VL_DESPESA,
       DT_DESPESA,
       TP_DESPESA,
       ID_CONTA
FROM TB_DESPESA
WHERE ID_USUARIO = [CÓDIGO DO USUÁRIO]
ORDER BY DT_DESPESA DESC, ID_DESPESA DESC;
```

8.2 Consultar todos os investimentos de um usuário, do mais recente ao mais antigo

A consulta inclui o nome do ativo para enriquecer a exibição na aplicação.

```
SELECT I.ID_INVESTIMENTO,
       I.DT_INVESTIMENTO,
       A.NM_ATIVO,
       A.TP_ATIVO,
       I.VL_INVESTIDO,
       I.QT_ATIVO,
       I.VL_UNITARIO,
       I.ST_INVESTIMENTO
FROM TB_INVESTIMENTO I
JOIN TB_ATIVO A
  ON A.ID_ATIVO = I.ID_ATIVO
WHERE I.ID_USUARIO = [CÓDIGO DO USUÁRIO]
ORDER BY I.DT_INVESTIMENTO DESC, I.ID_INVESTIMENTO DESC;
```

9. Consulta para o dashboard

A consulta abaixo utiliza ROW_NUMBER() para identificar a despesa e o investimento mais recentes de cada usuário. Depois, faz LEFT JOIN dessas duas visões com TB_USUARIO. O filtro final por ID_USUARIO garante que o resultado tenha no máximo uma linha para o usuário solicitado, mesmo que ele ainda não possua despesa ou investimento.

```
WITH ULTIMA_DESPESA AS (
    SELECT D.*,
           ROW_NUMBER() OVER (
               PARTITION BY D.ID_USUARIO
               ORDER BY D.DT_DESPESA DESC, D.ID_DESPESA DESC
           ) AS RN
    FROM TB_DESPESA D
),
ULTIMO_INVESTIMENTO AS (
    SELECT I.*,
           ROW_NUMBER() OVER (
               PARTITION BY I.ID_USUARIO
               ORDER BY I.DT_INVESTIMENTO DESC, I.ID_INVESTIMENTO DESC
           ) AS RN
    FROM TB_INVESTIMENTO I
)
SELECT U.ID_USUARIO,
       U.NM_USUARIO,
       U.DS_EMAIL,
       U.ST_USUARIO,
       U.TP_USUARIO,
       D.ID_DESPESA AS ID_ULTIMA_DESPESA,
       D.DS_DESPESA AS DS_ULTIMA_DESPESA,
       D.VL_DESPESA AS VL_ULTIMA_DESPESA,
       D.DT_DESPESA AS DT_ULTIMA_DESPESA,
       I.ID_INVESTIMENTO AS ID_ULTIMO_INVESTIMENTO,
       A.NM_ATIVO AS NM_ATIVO_ULTIMO_INVESTIMENTO,
       I.VL_INVESTIDO AS VL_ULTIMO_INVESTIMENTO,
       I.DT_INVESTIMENTO AS DT_ULTIMO_INVESTIMENTO,
       I.ST_INVESTIMENTO AS ST_ULTIMO_INVESTIMENTO
FROM TB_USUARIO U
LEFT JOIN ULTIMA_DESPESA D
       ON D.ID_USUARIO = U.ID_USUARIO
      AND D.RN = 1
LEFT JOIN ULTIMO_INVESTIMENTO I
       ON I.ID_USUARIO = U.ID_USUARIO
      AND I.RN = 1
LEFT JOIN TB_ATIVO A
       ON A.ID_ATIVO = I.ID_ATIVO
WHERE U.ID_USUARIO = [CÓDIGO DO USUÁRIO];
```

10. Validação dos comandos em relação ao enunciado

Item exigido	Seção	Status
Cadastrar novo usuário	5.1	Atendido
Cadastrar conta bancária	5.2	Atendido
Cadastrar receita	5.3	Atendido
Cadastrar despesa	5.4	Atendido
Cadastrar investimento	5.5	Atendido
Alterar usuário por ID	6.1	Atendido
Alterar receita por usuário + receita	6.2	Atendido
Alterar despesa por usuário + despesa	6.3	Atendido
Alterar investimento por usuário + investimento	6.4	Atendido
Consultar usuário por ID	7.1	Atendido
Consultar despesa por usuário + despesa	7.2	Atendido
Consultar investimento por usuário + investimento	7.3	Atendido
Despesas ordenadas por data DESC	8.1	Atendido
Investimentos ordenados por data DESC	8.2	Atendido
Dashboard com uma linha, última despesa e	9	Atendido

Item exigido	Seção	Status
último investimento		
Integridade do modelo evoluído	3.1	Atendido - cardinalidades, PK/FK, UNIQUE e CHECK
DDL Oracle das novas tabelas	3.2	Atendido - CREATE TABLE executável
Justificativa de IDs e valor investido	3.3	Atendido - decisões documentadas

11. Conclusão

A solução mantém a identidade e a modelagem central da SignalInvest, mas evolui o banco para representar claramente as operações financeiras solicitadas na atividade. Além dos comandos DML e das consultas, a versão final documenta as cardinalidades, as restrições de integridade e os scripts DDL Oracle das novas tabelas. Todos os comandos utilizam máscaras de substituição nos valores dinâmicos, filtros por identificadores e nomenclatura coerente com o modelo físico do projeto, mantendo a solução preparada para futura integração com Java/JDBC.